메멘토애니 (Memento Ani) 프로젝트 컨텍스트

프로젝트 개요

반려동물과 함께하는 모든 순간을 기록하고, 이별 후에도 따뜻한 추억을 간직할 수 있는 **메모리얼 커뮤니티 플랫폼**

핵심 컨셉

- My Little Puppy 게임 감성 (밝고 따뜻한 파스텔톤)
- 듀얼 모드: 일상(active) ↔ 추모(memorial)
- 직접적 죽음/천국 언급 대신 "무지개다리" 같은 완곡한 표현

기술 스택

영역	기술
Frontend	Next.js 14 (App Router), TypeScript, Tailwind CSS
UI	Radix UI + shadcn/ui
Backend	Supabase (PostgreSQL, Auth, Storage)
AI	OpenAI GPT-4o-mini
배포	Vercel (예정)

프로젝트 구조

```
src/
├── app/
│   ├── page.tsx              # 메인 라우터 (탭 기반 SPA)
│   ├── api/
│   │   ├── chat/route.ts     # AI 펫톡 API
│   │   └── reminders/        # 리마인더 CRUD API
│   └── auth/callback/        # Supabase Auth 콜백
├── components/
│   ├── pages/                # 8개 메인 페이지 컴포넌트
│   ├── common/               # Layout, Footer
│   ├── features/             # 기능별 컴포넌트
│   ├── modals/               # 모달 컴포넌트
│   └── ui/                   # shadcn/ui 기본 컴포넌트
├── contexts/
│   ├── AuthContext.tsx       # 인증 상태 관리
│   └── PetContext.tsx        # 반려동물 데이터 관리
├── lib/
│   ├── agent.ts              # AI 에이전트 로직
│   ├── supabase.ts           # Supabase 클라이언트 (클라이언트용)
│   ├── supabase-server.ts    # Supabase 서버 클라이언트 (API용, 세션 인증)
│   └── storage.ts            # 스토리지 유틸
└── types/index.ts            # 모든 타입 정의 (중앙 관리)
```

핵심 컨벤션

1. 타입 관리

```
// ✅ 올바른 방법: types/index.ts에서 import
import { Pet, PetPhoto, TabType } from "@/types";

// ❌ 잘못된 방법: 컴포넌트에서 자체 정의
interface Pet { ... }  // 금지!
```

2. 컬러 시스템

```
일상 모드: #05B2DC (하늘색), #E0F7FF (연하늘), #38BDF8 (밝은파랑)
추모 모드: amber 계열 (황금빛, 따뜻한 느낌)
배경: 뭉게구름 화이트 그라데이션
```

3. 코드 스타일

- 이모지 사용 금지 (서비스 특성상)
- 컴포넌트 파일 상단에 JSDoc 주석으로 설명
- API 응답은 camelCase로 변환해서 사용

주요 페이지 (8개 탭)

탭	컴포넌트	상태
홈	HomePage	✅ 완료
우리의 기록	RecordPage	✅ 완료
커뮤니티	CommunityPage	🟡 목업
AI 펫톡	AIChatPage	✅ 완료
입양정보	AdoptionPage	🟡 목업
지역정보	LocalPage	🟡 목업
분실동물	LostPage	🟡 목업
펫매거진	MagazinePage	🟡 목업

듀얼 모드 시스템

```
// Pet 상태에 따라 UI/기능 분기
if (selectedPet?.status === "memorial") {
    // 추모 모드: 황금빛 테마, 치유 게시판, AI 위로 상담
} else {
```

```
    // 일상 모드: 하늘색 테마, 케어 알림, AI 케어 매니저
}
```

데이터베이스 스키마 (Supabase)

핵심 테이블

- `pets` - 반려동물 정보
- `pet_media` - 사진/영상
- `timeline_entries` - 타임라인 일기
- `pet_reminders` - 케어 알림
- `chat_messages` - AI 대화 기록
- `pet_memories` - AI 장기 메모리

RLS 정책

모든 테이블에 `auth.uid() = user_id` 정책 적용

현재 상태 및 우선순위

완료된 기능 ✅

- 반려동물 CRUD + 사진/영상 관리
- AI 펫톡 (일상/추모 모드, 감정 분석)
- 타임라인 일기 + 편집
- 케어 리마인더 시스템
- 듀얼 모드 UI 전환
- 동물별 게시판 분류
- **API 보안**: reminders API 세션 기반 인증 완료 (supabase-server.ts)
- **관리자 페이지**: AdminPage.tsx (사용자/게시물 관리, 대시보드)
- **프리미엄 모달**: PremiumModal.tsx (커피 한 잔 값 설득)
- **로그인 프롬프트**: LoginPromptModal.tsx (비로그인 유저 유도)
- **랜딩 페이지 제거**: 비로그인자도 홈 화면 바로 접근 가능
- **isPremium DB 연동**: AuthContext에서 profiles.is_premium + premium_expires_at 체크
- **커뮤니티 DB 연동**: /api/posts CRUD, 5개 게시판 분류, 좋아요/댓글/신고
- **입양정보 DB 연동**: /api/adoption 공공데이터 연동 + 폴백 목업
- **분실동물 DB 연동**: /api/lost-pets 완전 CRUD + 이미지 업로드
- **펫매거진 DB 연동**: /api/magazine 관리자 작성 + 본문 이미지 삽입
- **지역정보 DB 연동**: /api/local-posts CRUD + 지역 필터
- **접근성**: 모달 19개 aria 속성 (role, aria-modal, aria-labelledby)
- **컴포넌트 분리**: RecordPage에서 PetProfileCard, PetPhotoAlbum 추출
- **개인화**: 온보딩 데이터 기반 홈페이지 HERO 개인화 (planning/current/memorial)
- **레벨 아이콘**: petType별 (dog/cat/other) 7단계 아이콘 시스템
- **API 엔드포인트 상수화**: src/config/apiEndpoints.ts

개선 필요 🔧

1. **결제 연동**: 포트원(PortOne) 연동 - 프리미엄 구독 실결제
2. **대형 컴포넌트 분리**: AIChatPage(1408줄), LostPage(1356줄) 서브컴포넌트 추출
3. **API URL 마이그레이션**: 기존 하드코딩 URL → apiEndpoints.ts 상수 사용으로 점진 전환

주의사항

하지 말 것

- 컴포넌트에서 자체 타입 정의 (types/index.ts 사용)
- 직접적 죽음/사망 표현 (무지개다리, 이곳 사용)
- 이모지 남용 (서비스 톤앤매너)
- console.log 프로덕션에 남기기

반드시 할 것

- 에러 처리 + 사용자 피드백
- 모바일 반응형 고려
- 모드별 테마 색상 적용
- 타입 안전성 확보

자주 발생하는 실수 & 디버깅

1. 같은 기능 버튼 여러 개 → 일부만 수정

문제: "새 반려동물" 버튼이 3군데 있는데 2군데만 handleAddNewPet으로 수정 **해결**: 수정 전 반드시 grep으로 전체 검색

```
grep -n "setIsPetModalOpen\|handleAddNewPet" src/components/pages/RecordPage.tsx
```

2. Dynamic Import 사용 금지

원칙: 즉각적인 반응이 UX에 더 중요 (스포티파이 사례) **해결**: 모달, 컴포넌트는 일반 import 사용. dynamic import 사용하지 않음

3. 반응형 텍스트 줄바꿈

문제: "특별해지는 곳"이 "특" / "별해지는 곳"으로 끊김 **해결**: 짧은 텍스트로 변경 ("특별한 매일을 함께")

4. Context 로딩 타이밍

문제: pets.length가 0으로 나옴 (데이터 로딩 전) **해결**: isLoading 체크 후 렌더링, useEffect 의존성 확인

5. 프리미엄 모달 안 뜸

문제: 제한 체크 로직이 있는 함수를 안 쓰고 직접 setModal(true) 호출 **해결**: 항상 래퍼 함수 (handleAddNewPet) 사용

프리미엄/무료 회원 제한

기능	무료	프리미엄 (월 7,900원)
AI 펫톡	하루 10회	무제한
반려동물 등록	1마리	무제한
사진 저장	100장	무제한

```
// config/constants.ts — 중앙 관리
export const FREE_LIMITS = { PETS: 1, PHOTOS: 100, DAILY_CHAT: 10 };

// AuthContext.tsx — DB 기반 프리미엄 체크 (완료)
const isPremium = data?.is_premium && (!expiresAt || new Date(expiresAt) > new
Date());
```

관리자 시스템

```
// types/index.ts
export const ADMIN_EMAILS = ["sharkwind1@gmail.com"];

export function isAdmin(email?: string | null): boolean {
    if (!email) return false;
    return ADMIN_EMAILS.includes(email);
}

// Layout.tsx에서 관리자 탭 조건부 표시
{isAdmin(user?.email) && <AdminTab />}
```

자주 수정하는 파일

```
src/types/index.ts           # 타입 추가/수정
src/contexts/PetContext.tsx # 펫 데이터 로직
src/app/api/chat/route.ts    # AI 프롬프트 수정
src/components/pages/*.tsx  # 각 페이지 UI
```

작업 방식: 서브에이전트 오케스트레이션 (필수)

> ***모든 작업은 이 방식으로 진행한다. 예외 없음.*** *상세 팀 구성과 호출 패턴은 AGENTS.md 참고.*

핵심 원칙

1. **우리는 팀이다** - 승빈님 포함 모든 에이전트가 동등한 동료
2. **권한 받으면 물어보지 말고 실행** - 승빈님이 자거나 외출 중이면 토큰 다 써서라도 끝내놓기
3. **망가지면 승빈님이 고치라고 한다** - 완벽하게 하려고 멈추지 말고 일단 해놓기
4. **디자인 작업은 반드시 3번(UX) + 4번(비주얼) 피드백 후 구현**
5. **모든 주요 결과물은 7번(비판적 사고 에이전트) 검증 필수**

팀 (8명)

번호	역할	핵심
1	프론트엔드 엔지니어	React/Next.js, 타입 안전성, 모바일 퍼스트
2	백엔드 엔지니어	Supabase, API, RLS, 보안
3	UX/UI 디자이너	사용자 흐름, 접근성, 감성

4	비주얼 디자이너 (원로)	컬러, 픽셀 아트, 브랜드 일관성
5	QA 엔지니어	빌드, 타입체크, 에지케이스
6	AI/프롬프트 엔지니어	AI 펫톡, 감정 분석, 톤앤매너
7	**비판적 사고 에이전트**	객관적 평가, 근거 기반 판단, 통과/수정/반려
8	PM/기획자	우선순위, MVP 범위, 사업적 임팩트

오케스트레이션 흐름

```
작업 시작 → 8번(PM) 우선순위 판단
          → 해당 전문 에이전트 투입 (Task 도구로 서브에이전트 호출)
          → 디자인이면 3번+4번 먼저
          → 구현 완료 → 5번(QA) + 7번(비판적 사고) 검증
          → 7번 통과 → 완료
          → 7번 반려 → 수정 후 재검증
```

서브에이전트 호출

- Claude Code의 Task 도구로 `general-purpose` 서브에이전트를 띄움
- 프롬프트에 역할 번호, 역할명, 판단 기준, 작업 내용을 명시
- 토론 필요 시 여러 에이전트를 **병렬로 동시 호출**
- 호출 상세 패턴은 AGENTS.md의 "서브에이전트 호출 방법" 섹션 참고

개발자 정보

- **이름**: 안승빈
- **상황**: 풀스택 부트캠프 수료 예정 (2026년 3월)
- **목표**: MVP 런칭 → 창업지원금 신청